MOTIBAKERY

Cake Selection & Order Management

Flutter Application

Complete Technical Documentation

Document Type	Flutter App — Technical Spec
Platform	Android (APK)
Framework	Flutter (Dart)
Target Roles	Counter Staff & Cake Room
Version	Phase 1 — MVP
Brand	Motibakery

1. Project Overview

This document provides the complete technical specification for the Motibakery Flutter Android application. The app serves as a digital cake ordering platform used in-store, replacing printed catalogues and enabling real-time coordination between the Counter team and the Cake Room.

What this app does

- ✦ Counter staff browse the cake gallery with customers and place customized orders
- ✦ Cake Room staff receive orders, track preparation, and update statuses
- ✦ Both roles receive real-time notifications via Firebase Cloud Messaging
- ✦ All product data, users, and pricing are managed from the Admin CMS (web)

1.1 App Goals

- Eliminate printed cake catalogues with a beautiful digital gallery
- Streamline order entry — flavour, weight, delivery date, images, notes
- Enable real-time status flow: Counter → Cake Room → Counter
- Enforce pricing rules automatically (flavour-based increments)
- Role-based access: Counter and Cake Room see different dashboards

1.2 Scope (Phase 1 — MVP)

Feature	Included in Phase 1	Notes
Cake Gallery (Digital Booklet)	✅Yes	Grid view with search & filter
Role-based Login	✅Yes	Counter & Cake Room roles
Order Placement (Counter)	✅Yes	Full form with pricing engine
Cake Room Dashboard	✅Yes	Queue + status updates
Push Notifications (FCM)	✅Yes	Order ready + new order alerts
Admin CMS	❌Web only	Not part of this Flutter app
KOT Printing	❌Phase 2	Thermal/Bluetooth printer
Payment Tracking	❌Phase 2	Billing module
Multi-branch	❌Phase 2	Future expansion
Analytics Dashboard	❌Phase 2	Admin analytics

2. User Roles & Access

The Flutter app supports two distinct roles. Both roles share the same APK — the app detects the user role after login and routes them to the appropriate dashboard. There is no public sign-up; all accounts are created exclusively via the Admin CMS.

2.1 Counter Role

Capability	Detail
Browse Cake Gallery	Full grid gallery with search, filter, and detail view
Place Orders	Select cake, flavour, weight, delivery date, add notes & image
View Order History	See all orders placed from this counter session
Receive Notifications	FCM alert when a cake is marked Prepared by Cake Room
Pricing Preview	Automatic price display based on weight + pricing rules

2.2 Cake Room Role

Capability	Detail
Order Queue View	See all incoming orders grouped by status
View Order Details	Full specs: cake design, flavour, weight, notes, image attachments
Update Status	Move order: New → In Progress → Prepared
Prepared Trigger	Marking Prepared sends instant FCM push to Counter
Optional Notifications	Receive FCM push when a new order is placed (configurable)

2.3 Authentication & Session

- Login screen is the app entry point for both roles
- No public registration — Admin CMS creates all accounts
- After login, role is fetched from backend and persisted locally
- Persistent login session using secure token storage (flutter_secure_storage)
- Session expires only on logout or token invalidation by Admin
- Wrong role assignment is handled server-side — mismatched tokens rejected

3. Screens & Navigation

The app is organized into two navigation trees based on role. Both share the Login screen. After authentication, the app branches into the Counter navigation tree or the Cake Room navigation tree.

3.1 App Navigation Structure

Below is the full screen map for Phase 1:

Counter Navigation Tree

```
[Login Screen]
    ↓
[Counter Home — Cake Gallery]
    ├── [Cake Detail Screen]
    │       └── [Place Order Screen]
    │               └── [Order Confirmation Screen]
    └── [My Orders Screen]
            └── [Order Detail View (read-only)]
```

Cake Room Navigation Tree

```
[Login Screen]
    ↓
[Cake Room Dashboard — Order Queue]
    └── [Order Detail Screen]
            └── [Update Status Actions]
```

3.2 Screen Inventory

Screen Name	Role	Key Purpose
Login Screen	Both	Email/password login, role detection
Counter Home (Cake Gallery)	Counter	Browse all active cakes in grid
Cake Detail Screen	Counter	View design, flavours, weight, pricing
Place Order Screen	Counter	Full order form with price calculator
Order Confirmation Screen	Counter	Order success + order ID display
My Orders Screen	Counter	List of orders placed in current session
Order Detail View (Counter)	Counter	Read-only view of a placed order

Cake Room Dashboard	Cake Room	Order queue: New / In Progress / Prepared
Order Detail Screen (Cake Room)	Cake Room	Full order specs + status update actions

4. Screen-by-Screen Specification

4.1 Login Screen

Entry point for all users. No role selector on screen — role is determined server-side from the authenticated account.

UI Elements

- Motibakery logo (centered, top area)
- Email input field
- Password input field (obscured, toggle to show)
- Login button (primary — orange brand color)
- Error message area (inline, below fields)
- Loading indicator on button during API call

Logic & Behavior

- On successful login → fetch role from backend → route to correct dashboard
- Token stored in flutter_secure_storage
- Failed login → display inline error (do not clear password field)
- Form validation: both fields required, email format validated
- Keyboard: email input uses emailAddress keyboard type
- Auto-focus email field on screen open

4.2 Counter Home — Cake Gallery

The main screen for Counter staff. Displays all active cakes as a visual grid. This is the digital replacement for the printed cake booklet.

UI Elements

- App bar with Motibakery branding and logout icon
- Search bar at top (persistent, debounced)
- Filter chips row: category/tag filters (horizontally scrollable)
- Grid view: 2 columns, card-based layout
- Each cake card shows: cake image (thumbnail), cake name, base price/weight range
- Pull-to-refresh support
- Empty state view if no cakes match filter/search
- Loading shimmer on first load

Cake Card Design

  - Rounded corners, subtle shadow

- Image takes ~60% of card height (AspectRatio 4:3)
- Cake name in bold below image
- Weight range and base rate in smaller text
- 'View Details' tap navigates to Cake Detail Screen

Search & Filter Logic

- Search is real-time (300ms debounce) — filters cake name client-side
- Category filters are fetched from backend on screen load
- Multiple filters can be active simultaneously
- Active filter chips highlighted in orange

4.3 Cake Detail Screen

Full detail view for a selected cake design. Customer can review all options before proceeding to order placement.

UI Elements

- Full-width cake image (hero animation from gallery card)
- Cake name (large heading)
- Description / design notes
- Flavour options: displayed as selectable chips (single-select)
- Weight range: Min — Max displayed in kg
- Base rate display (if set by Admin)
- Pricing note: 'Final price calculated at order' if flavour rules apply
- CTA button: 'Place Order' (fixed at bottom, full width)

Navigation

- Tapping 'Place Order' passes selected cake and default flavour to Place Order Screen
- Back button returns to Gallery with scroll position preserved

4.4 Place Order Screen

The core order entry form. Counter staff fill this with the customer while they are at the counter.

Form Fields

Field	Type	Validation
Cake Design	Pre-filled (from detail screen)	Read-only, cannot change
Flavour	Dropdown / chips selector	Required — must select one
Weight (kg)	Numeric input with slider	Required — within cake min/max range

Delivery Date	Date picker	Required — must be today or future date
Delivery Time	Time picker	Optional
Customer Name	Text input	Optional
Custom Notes	Multi-line text area	Optional — 300 char limit
Reference Image	Image picker (gallery/camera)	Optional — max 5MB

Price Calculator Widget

- Displayed prominently below form fields
- Base price = base rate × weight
- Flavour increment applied on top (from pricing rules)
- Total updates live as weight or flavour changes
- Format: ₹ 1,200.00 (Indian Rupee, 2 decimal places)

Image Attachment

- Camera option: opens device camera
- Gallery option: opens image picker
- Selected image shown as thumbnail with remove (X) button
- Only one image per order in Phase 1
- Image uploaded to cloud storage on form submission, URL stored in order

Submission Logic

- Validate all required fields before submission
- Show loading overlay during API call
- On success → navigate to Order Confirmation Screen
- On failure → show error snackbar, keep form data intact
- Prevent double submission with button disable during loading

4.5 Order Confirmation Screen

Displayed after a successful order placement. Provides order reference and summary.

UI Elements

- Success icon (animated checkmark — orange)
- 'Order Placed Successfully!' heading
- Order ID (bold, copyable)
- Order summary: cake name, flavour, weight, delivery date, total price
- 'Place Another Order' button → back to Gallery
- 'View My Orders' button → My Orders Screen

4.6 My Orders Screen

Lists all orders placed from the current Counter session. Shows status in real-time.

Order List Item

- Order ID and cake name
- Delivery date and weight
- Status badge: New (grey) / In Progress (blue) / Prepared (green)
- Tap to open read-only Order Detail View

Real-time Updates

- Status updates refresh via polling (30 second interval) or FCM push
- When status changes to Prepared — badge animates and notification shown

4.7 Cake Room Dashboard — Order Queue

The primary screen for Cake Room staff. Displays all orders organized by preparation status using a tab layout.

Tab Structure

- Tab 1: New Orders — all orders awaiting start
- Tab 2: In Progress — orders currently being prepared
- Tab 3: Prepared — completed orders awaiting counter pickup

Order Card in Queue

- Order ID and cake name (prominent)
- Customer name (if provided)
- Delivery date & time
- Flavour and weight
- Time since order was placed
- Quick action: tap card → full Order Detail Screen

Sorting & Priority

- New Orders: sorted by delivery date (earliest first)
- In Progress: sorted by time entered (FIFO)
- Prepared: sorted by most recently prepared
- Visual indicator if delivery date is today or overdue (orange/red highlight)

4.8 Order Detail Screen — Cake Room

Full order detail view for Cake Room. Includes all specifications needed to prepare the cake and the action to update status.

Content Sections

- Order Header: Order ID, placed at time, current status badge
- Cake Details: cake name, design image thumbnail
- Preparation Specs: flavour, weight (kg), custom notes
- Customer Reference: customer name, delivery date & time
- Reference Image: full-width display if attached (tap to zoom)
- Pricing: shown for reference (not editable)

Status Action Buttons

Current Status	Available Action	Outcome
New	'Start Preparation' button	Status → In Progress
In Progress	'Mark as Prepared' button	Status → Prepared + FCM to Counter
Prepared	No action available	Read-only view

Status Update Logic

- Tapping action button shows confirmation bottom sheet
- On confirm → API call to update status
- On Prepared → backend triggers FCM push notification to Counter role
- Loading state on button during API call
- On success → queue list refreshes, order moves to correct tab

5. Push Notifications (FCM)

Firebase Cloud Messaging (FCM) handles all real-time push notifications. The backend sends FCM messages via server-to-device targeting.

5.1 Notification Types

Trigger	Recipient	Title	Body	Action on Tap
Order Placed	Cake Room (optional)	New Order Received	Order #ID — CakeName — DeliveryDate	Open Order Detail
Order Prepared	Counter	Order Ready! □	Order #ID is ready for pickup	Open My Orders

5.2 FCM Setup in Flutter

- Package: firebase_messaging
- Request notification permission on first launch (Android 13+)
- Handle foreground messages → show in-app banner (flutter_local_notifications)
- Handle background/terminated messages → system notification tray
- On notification tap → deep link to relevant screen based on payload
- FCM token registered to backend on login, cleared on logout

Notification Payload Structure

```
{ "order_id": "ORD-001", "type": "order_prepared", "cake_name": "Black Forest" }
```

6. Pricing Rule Engine

Pricing is calculated client-side using rules fetched from the backend. The admin configures these rules in the CMS. The Flutter app fetches and caches them at launch.

6.1 Pricing Formula

```
Total Price = (Base Rate + Flavour Increment) × Weight
```

6.2 Rule Configuration (from Admin CMS)

Rule Type	Example	Applied When
Flavour Increment (flat)	Chocolate: +₹50/kg	Selected flavour matches rule
Flavour Increment (%)	Truffle: +10%	Applied to base rate before × weight
Category Adjustment	Premium category: +₹100/kg	Cake belongs to tagged category
No Rule	Vanilla: +₹0	Default — only base rate applies

6.3 Client-side Implementation

- Pricing rules fetched on login → stored in local state (Provider/Riverpod)
- Rules refreshed if app has been backgrounded for > 30 minutes
- Price recalculated instantly on every form field change
- If no rule matches the selected flavour → price = base rate × weight only
- Displayed price is always rounded to 2 decimal places

7. Technical Architecture

7.1 Technology Stack

Layer	Technology	Purpose
UI Framework	Flutter (Dart)	Cross-platform UI — Android APK
State Management	Riverpod (or Provider)	App-wide state, role, orders, gallery
HTTP Client	Dio	API calls with interceptors, retry logic
Push Notifications	firebase_messaging	FCM push delivery
Local Notifications	flutter_local_notifications	In-app notification banners
Secure Storage	flutter_secure_storage	Auth token persistence
Image Picker	image_picker	Camera & gallery access
Date/Time Picker	Custom Flutter widget	Delivery date & time selection
Image Cache	cached_network_image	Efficient remote image loading

7.2 Project Folder Structure

```
lib/
  ├── main.dart                  # App entry, Firebase init, routing
  ├── app/
  │   ├── router.dart            # GoRouter — role-based route guards
  │   └── theme.dart             # Brand colors, typography, components
  ├── features/
  │   ├── auth/                  # Login screen, auth logic, token storage
  │   ├── gallery/               # Cake Gallery screen + Cake Detail screen
  │   ├── orders/                # Place Order, Confirmation, My Orders
  │   └── cake_room/             # Cake Room Dashboard, Order Detail
  ├── shared/
  │   ├── models/                # Cake, Order, PricingRule, User models
  │   ├── services/              # API service, FCM service, image upload
  │   ├── providers/             # Riverpod providers
  │   └── widgets/               # Reusable UI components
  └── utils/
      ├── price_calculator.dart  # Pricing rule engine logic
      └── validators.dart        # Form validators
```

7.3 State Management Approach

- Auth State: global — user role, token, login status
- Gallery State: cached list of cakes + filter/search state
- Pricing Rules: global cache, refreshed periodically
- Order Form State: local to Place Order screen — disposed on exit
- Cake Room Queue: polled every 60s + FCM-triggered refresh

7.4 API Communication

- Base URL configured per environment (dev / staging / production)
- Auth token sent in Authorization: Bearer <token> header on all requests
- Dio interceptor auto-refreshes token or logs out on 401 response
- Offline state: show cached data with 'You are offline' banner
- All API errors parsed into user-friendly messages

Key API Endpoints (Flutter will consume)

Method	Endpoint	Used By	Description
POST	/auth/login	Both	Login, returns token + role
GET	/cakes	Counter	Fetch all active cakes with filters
GET	/cakes/:id	Counter	Fetch single cake detail
GET	/pricing-rules	Counter	Fetch all active pricing rules
POST	/orders	Counter	Create a new order
GET	/orders/my	Counter	Fetch current user's orders
GET	/orders/queue	Cake Room	Fetch order queue by status
GET	/orders/:id	Both	Fetch single order detail
PATCH	/orders/:id/status	Cake Room	Update order status
POST	/upload/image	Counter	Upload reference image
POST	/fcm/register	Both	Register FCM device token

8. Data Models

Below are the core data models the Flutter app will use. These mirror the backend API response structures.

8.1 Cake Model

```
class Cake {
  final String id;
  final String name;
  final String imageUrl;
  final String? description;
  final List<String> flavours;       // e.g. ['Vanilla', 'Chocolate', 'Truffle']
  final double minWeight;            // in kg
  final double maxWeight;            // in kg
  final double? baseRate;            // price per kg
  final List<String> categories;     // tags for filtering
  final bool isActive;
}
```

8.2 Order Model

```
class Order {
  final String id;
  final String cakeId;
  final String cakeName;
  final String flavour;
  final double weight;               // kg
  final DateTime deliveryDate;
  final DateTime? deliveryTime;
  final String? customerName;
  final String? notes;
  final String? imageUrl;            // reference image
  final double totalPrice;
  final OrderStatus status;          // new | in_progress | prepared
  final DateTime createdAt;
  final String createdBy;            // counter user ID
}

enum OrderStatus { newOrder, inProgress, prepared }
```

8.3 Pricing Rule Model

```
class PricingRule {
  final String id;
  final String? flavour;             // null = applies to all flavours
  final String? category;            // null = not category-based
  final double incrementAmount;      // flat amount per kg
  final double? incrementPercent;    // % on base rate (alt to flat)
  final bool isActive;
}
```

9. UI Design System

The app uses Motibakery's brand identity — warm orange, clean whites, and bold typography. The design feels premium and approachable, suited for an in-store customer-facing screen.

9.1 Brand Colors

Color Name	Hex	Usage
Brand Orange	#D94F1E	Primary buttons, active states, icons, headings
Accent Orange	#F28B5B	Secondary elements, hover states, price display
Dark Charcoal	#2B2B2B	Body text, headings on white background
Light Cream	#FFF5F2	Page backgrounds, card fills
Mid Grey	#7A7A7A	Secondary text, placeholders, subtitles
White	#FFFFFF	Card surfaces, modal backgrounds
Success Green	#4CAF50	Prepared status badge
Info Blue	#2196F3	In Progress status badge
Neutral Grey	#9E9E9E	New status badge

9.2 Typography

Style	Font	Size	Weight	Usage
Display	Inter / Poppins	28sp	Bold	Screen titles, order confirmation
Heading	Inter / Poppins	20sp	SemiBold	Section headers, cake names
Sub-heading	Inter / Poppins	16sp	Medium	Card labels, field labels
Body	Inter / Poppins	14sp	Regular	Descriptions, order details
Caption	Inter / Poppins	12sp	Regular	Timestamps, secondary info
Price Display	Inter / Poppins	22sp	Bold	Price in order form
Button	Inter / Poppins	16sp	SemiBold	All CTA buttons

9.3 Component Library

Primary Button

- Background: Brand Orange (#D94F1E)
- Text: White, 16sp, SemiBold
- Border radius: 12px

- Height: 52px, Full width in forms
- Loading state: CircularProgressIndicator inside button

Status Badge

- New: Grey background, dark text — 'New'
- In Progress: Blue background, white text — 'In Progress'
- Prepared: Green background, white text — 'Prepared ✓'
- Shape: Rounded pill, 6px border radius, 8px horizontal padding

Cake Card (Gallery)

- White background, 12px border radius, subtle drop shadow
- Image: AspectRatio 4:3, cover fit, rounded top corners
- Padding: 12px all sides for text area
- Cake name: 16sp bold, 1 line max with ellipsis
- Weight range: 12sp grey
- Base rate: 14sp orange, right-aligned

Order Form Fields

- Outlined text fields with orange focus border
- Label floats on focus
- Error text: red, appears below field
- Field height: 52px for single-line, 120px for multi-line notes

10. Development Checklist

Use this checklist to track implementation progress for Phase 1 MVP.

10.1 Setup & Configuration

Task	Notes
Create Flutter project	flutter create motibakery_app
Configure Firebase project	Firebase Console — add Android app, download google-services.json
Add all dependencies to pubspec.yaml	See Section 7.1 for full package list
Configure app theme	Brand colors, typography — Section 9
Setup GoRouter with role-based guards	Redirect unauthenticated users to Login
Configure Dio HTTP client	Base URL, auth interceptor, error handler
Setup Riverpod providers	AuthProvider, GalleryProvider, OrderProvider, PricingProvider
Configure flutter_secure_storage	Token persistence across app restarts

10.2 Feature Implementation Order (Recommended)

#	Feature	Dependencies
1	Login Screen + Auth Flow	Backend auth API, Dio, secure_storage
2	Cake Gallery Screen	Cakes API, image cache
3	Cake Detail Screen	Gallery (no extra deps)
4	Pricing Rule Engine	Pricing rules API
5	Place Order Screen	Orders API, image_picker, image upload API
6	Order Confirmation Screen	Orders (no extra deps)
7	My Orders Screen	My orders API
8	Cake Room Dashboard	Order queue API, tabs
9	Order Detail + Status Update	Status update API
10	FCM Push Notifications	Firebase, FCM token registration API
11	Offline Handling & Polish	Connectivity, shimmer loading, error states

10.3 Quality Checklist

- All forms validated before submission
- Loading states on all API calls

- Error states with user-friendly messages
- Empty states for gallery, orders list, queue
- Pull-to-refresh on all list screens
- Keyboard dismissal on tap outside input
- Back button behavior consistent across all screens
- Images compressed before upload (max 1MB)
- FCM foreground, background, and terminated state all handled
- Test on Android 10, 11, 12, 13
- Test login with Counter role and Cake Room role accounts
- Test pricing calculator with edge cases: min weight, max weight, no flavour rule

11. pubspec.yaml Dependencies

Below is the full list of recommended Flutter packages for this project:

```yaml
dependencies:
  flutter:
    sdk: flutter

  # State Management
  flutter_riverpod: ^2.x.x

  # Navigation
  go_router: ^13.x.x

  # Networking
  dio: ^5.x.x

  # Firebase
  firebase_core: ^2.x.x
  firebase_messaging: ^14.x.x

  # Local Notifications
  flutter_local_notifications: ^16.x.x

  # Secure Storage
  flutter_secure_storage: ^9.x.x

  # Image Handling
  image_picker: ^1.x.x
  cached_network_image: ^3.x.x

  # UI Utilities
  shimmer: ^3.x.x          # Loading placeholders
  intl: ^0.19.x            # Date/currency formatting
  gap: ^3.x.x              # SizedBox shorthand

  # Utilities
  connectivity_plus: ^5.x.x
  equatable: ^2.x.x
```

12. Next Steps & Backend Discussion

This document covers the complete Flutter application specification. Once the Flutter app development begins, the following backend decisions need to be finalized:

Topic	Decision Needed
Backend Framework	Node.js (Express/Fastify) vs Firebase Functions
Database	Firestore vs PostgreSQL (Supabase/Neon)
Image Storage	Firebase Storage vs Cloudinary vs S3
Auth System	Firebase Auth vs custom JWT
FCM Trigger	Backend function vs Firebase Cloud Functions
Deployment	Cloud Run vs Firebase Hosting + Functions vs VPS
API Design	REST vs GraphQL (REST recommended for this scope)

Document Status

This document covers Phase 1 (MVP) Flutter app only.

Backend API documentation will be created separately.

Admin CMS (web) documentation to follow after Flutter scope is confirmed.

Phase 2 features (KOT printing, billing, multi-branch) to be documented before Phase 2 kickoff.